

06 · Práctica integradora — Sistema de recomendación

Aplicación a un caso de negocio real de la tienda virtual: a partir del catálogo de productos y el historial de compras de cada cliente, generar un top-N de recomendaciones.

Productos --descripción--> embedding
Cliente --promedio de lo que compró--> perfil
Perfil --coseno contra lo NO comprado--> Top-N

El mismo `SentenceTransformer` de `05_transformers-sbert.ipynb` sirve para vectorizar productos; lo nuevo es cómo se agregan varios vectores de producto en **un** vector de cliente.

```
import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer

productos = pd.read_csv("../data/productos.csv")
clientes = pd.read_csv("../data/clientes.csv")
historial = pd.read_csv("../data/historial_de_compras.csv")

MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
sbert = SentenceTransformer(MODEL_NAME)

texto_producto = productos["nombre_producto"] + ". " + productos["descripcion"]
embeddings_producto = sbert.encode(texto_producto.tolist(), convert_to_numpy=True,
normalize_embeddings=True)
id_a_indice = {pid: i for i, pid in enumerate(productos["id_producto"])}

print(f"{len(productos)} productos vectorizados en {embeddings_producto.shape[1]} dimensiones")
```

```
/private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/venv311/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 42027.52it/s]
10 productos vectorizados en 384 dimensiones
```

Perfil de cliente: promedio de lo que compró

Cada compra aporta el embedding de su producto; el perfil del cliente es el promedio de esos vectores (una compra repetida pesa más, porque aparece más veces en el promedio).

```
def perfil_cliente(id_cliente):
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    indices = [id_a_indice[pid] for pid in compras]
    vector = embeddings_producto[indices].mean(axis=0)
    return vector / np.linalg.norm(vector), set(compras)
```

```
def recomendar(id_cliente, top_n=3, agregacion=perfil_cliente):
    vector_cliente, comprados = agregacion(id_cliente)
    similitudes = embeddings_producto @ vector_cliente
    ranking = [
        (productos.loc[i, "nombre_producto"], productos.loc[i, "categoria"],
        round(float(similitudes[i]), 3))
        for i in similitudes.argsort()[::-1]
        if productos.loc[i, "id_producto"] not in comprados
    ]
    return ranking[:top_n]
```

Dos clientes muy distintos

- **Cliente 1** compró casi solo electrónica: 5 laptops, 3 smartphones, 1 auriculares, más un par de prendas.
- **Cliente 3** compró en las 5 categorías del catálogo, pero con las zapatillas repetidas 5 de sus 11 compras.

```
for id_cliente in [1, 3]:
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    categorias = productos.set_index("id_producto").loc[compras, "categoria"]
    print(f"Cliente {id_cliente} compró: {sorted(categorias.unique())}")
    print("  Top-3 recomendado:", recomendar(id_cliente))
    print()
```

Cliente 1 compró: ['Electrónica', 'Ropa']

Top-3 recomendado: [('Mochila Urbana Tech', 'Ropa', 0.625), ('Cámara de Acción 4K', 'Electrónica', 0.575), ('Set de Ollas Premium', 'Hogar', 0.434)]

Cliente 3 compró: ['Deportes', 'Electrónica', 'Hogar', 'Libros', 'Ropa']

Top-3 recomendado: [('Camiseta Deportiva Ultralight', 'Ropa', 0.741), ('Auriculares Inalámbricos X', 'Electrónica', 0.399), ('Cámara de Acción 4K', 'Electrónica', 0.384)]

El resultado del cliente 1 sorprende a primera vista: el top-1 es "Mochila Urbana Tech", catalogada como **Ropa**, no como electrónica. Pero su descripción incluye "funda acolchada para laptops" — el embedding lee el contenido semántico, no la etiqueta de categoría, y encuentra una prenda genuinamente relevante para alguien que compró cinco laptops. La recomendación #2, una cámara de acción, sí es electrónica pura.

El problema de la agregación

La intuición sería que el cliente 3, al comprar en cinco categorías, tendría un perfil "diluido y sin dirección". Los números muestran algo distinto: el promedio queda dominado por lo que más se repite, no por la variedad.

```
categoria_de = productos.set_index("id_producto")["categoria"]
compras_cliente_3 = historial.loc[historial["id_cliente"] == 3, "id_producto"]
print(compras_cliente_3.map(categoria_de).value_counts())
```

```
id_producto
Ropa          6
Libros        2
Electrónica   1
Hogar         1
Deportes      1
Name: count, dtype: int64
```

```
vector_cliente_3, _ = perfil_cliente(3)
```

```
promedio_por_categoria = {}
for categoria in productos["categoria"].unique():
    indices = productos.index[productos["categoria"] == categoria]
    v = embeddings_producto[indices].mean(axis=0)
    promedio_por_categoria[categoria] = v / np.linalg.norm(v)

for categoria, v in promedio_por_categoria.items():
    print(f" cos(perfil cliente 3, centroide {categoria:12}) = {vector_cliente_3 @ v:.3f}")
```

```
cos(perfil cliente 3, centroide Electrónica ) = 0.513
cos(perfil cliente 3, centroide Ropa          ) = 0.916
cos(perfil cliente 3, centroide Libros        ) = 0.333
cos(perfil cliente 3, centroide Hogar         ) = 0.565
cos(perfil cliente 3, centroide Deportes      ) = 0.576
```

"Ropa" se despega con 0.916, muy por encima de cualquier otra categoría — no porque el cliente sea sobre todo comprador de ropa, sino porque las zapatillas, repetidas 5 veces, pesan 5 veces más que el libro o el balón que compró una sola vez. El promedio trata cada **compra** como un voto, no cada **interés**: un cliente explorador con una compra repetida termina pareciéndose, en el vector, a un cliente enfocado en esa única categoría.

Una alternativa: similitud al producto más cercano, no al promedio

En vez de mezclar todos los vectores comprados en uno solo (donde la repetición pesa), se puede recomendar por el producto **individual** que más se parece a cada candidato — cada compra vota una sola vez, sin importar cuántas veces se repitió.

```
def perfil_por_item_mas_cercano(id_cliente):
    compras = historial.loc[historial["id_cliente"] == id_cliente, "id_producto"]
    indices = [id_a_indice[pid] for pid in compras]
    return embeddings_producto[indices], set(compras)

def recomendar_por_item(id_cliente, top_n=3):
    vectores_comprados, comprados = perfil_por_item_mas_cercano(id_cliente)
    similitudes = (embeddings_producto @ vectores_comprados.T).max(axis=1) # mejor
    coincidencia individual
    ranking = [
        (productos.loc[i, "nombre_producto"], productos.loc[i, "categoria"],
        round(float(similitudes[i]), 3))
        for i in similitudes.argsort()[::-1]
        if productos.loc[i, "id_producto"] not in comprados
    ]
    return ranking[:top_n]
```

```
print("Cliente 3 – promedio de perfil:", recomendar(3))
print("Cliente 3 – mejor coincidencia individual:", recomendar_por_item(3))
```

Cliente 3 – promedio de perfil: [('Camiseta Deportiva Ultralight', 'Ropa', 0.741), ('Auriculares Inalámbricos X', 'Electrónica', 0.399), ('Cámara de Acción 4K', 'Electrónica', 0.384)]

Cliente 3 – mejor coincidencia individual: [('Camiseta Deportiva Ultralight', 'Ropa', 0.718), ('Smartphone Nexus 5G', 'Electrónica', 0.509), ('Cámara de Acción 4K', 'Electrónica', 0.501)]

El top-1 no cambia (la camiseta deportiva sigue siendo, con cualquiera de los dos criterios, el producto más cercano a algo que el cliente ya compró), pero el top-2 sí: pasa de "Auriculares" a "Smartphone Nexus 5G", más alineado con el resto del historial. El criterio por máximo evita que una compra repetida "vote" varias veces, sin necesidad de descartar información.

Entregable: top-N por cliente, con el criterio justificado

Se usa **promedio de perfil** como criterio por defecto — es más simple y, en un catálogo con miles de productos, diluye mejor el ruido de una compra aislada. Pero el experimento anterior deja una advertencia concreta: el promedio no pondera por *interés*, pondera por *cantidad de compras*, así que un cliente que solo repitió un producto puede terminar con un perfil casi idéntico al de alguien enfocado en esa categoría desde el principio. **Mejor coincidencia individual** es preferible cuando el historial mezcla categorías y las compras repetidas no deben dominar el perfil.

Preguntas abiertas que quedan sin resolver por diseño (no tienen una respuesta única, dependen del negocio):

- ¿Deduplicar antes de promediar, para que cada producto distinto vote una sola vez sin importar cuántas veces se compró?
- ¿Ponderar las compras recientes más que las antiguas?
- ¿Cómo evaluar la calidad de las recomendaciones sin datos futuros (¿A/B testing? ¿ocultar la última compra y ver si el modelo la recupera?)
- Un cliente **nuevo sin historial** no tiene perfil que promediar — el método no cubre ese caso (*cold start*); ahí se recurre típicamente a recomendar lo más vendido o pedir preferencias explícitas al registrarse.

```
tabla_final = []
for id_cliente in clientes["id_cliente"]:
    if (historial["id_cliente"] == id_cliente).sum() == 0:
        continue
    top1 = recomendar(id_cliente, top_n=1)
    if top1:
        nombre, categoria, score = top1[0]
        tabla_final.append({"id_cliente": id_cliente, "recomendación #1": nombre,
"categoria": categoria, "similitud": score})

pd.DataFrame(tabla_final).head(10)
```

	id_cliente	recomendación #1	categoría	similitud
0	1	Mochila Urbana Tech	Ropa	0.625
1	2	Mochila Urbana Tech	Ropa	0.558
2	3	Camiseta Deportiva Ultralight	Ropa	0.741
3	4	Camiseta Deportiva Ultralight	Ropa	0.696
4	5	Camiseta Deportiva Ultralight	Ropa	0.512
5	6	Mochila Urbana Tech	Ropa	0.637
6	7	Mochila Urbana Tech	Ropa	0.564
7	8	Camiseta Deportiva Ultralight	Ropa	0.749
8	9	Camiseta Deportiva Ultralight	Ropa	0.638
9	10	Camiseta Deportiva Ultralight	Ropa	0.493

Siguiente: `07_sintesis-comparativa.ipynb` — las tres representaciones de esta sesión, una al lado de la otra, y cómo elegir entre ellas.